
Data Structures

BS (CS) _Fall_2025

LAB 01 Tasks



Lab Overview

This lab focuses on understanding and implementing function templates in C++ along with unit testing using Google Tests. Students will start with a brief introduction of templates and progress to implementing functions using templates and validating them using Google Tests.

Task 0: Template Function Usage (Warm-up)

Given Code:

Students are provided with the following template functions:

```
template<typename T>
T add(T a, T b) {
    return a + b;
}

template<typename T>
T subtract(T a, T b) {
    return a - b;
}

template<typename T>
T multiply(T a, T b) {
    return a * b;
}

template<typename T>
T divide(T a, T b) {
    return a / b;
}
```

Student Task:

Write a `main()` function that:

1. Tests each function with at least 3 different data types (int, float, double)
2. Demonstrates both explicit template instantiation (`add<int>(5, 3)`) and implicit instantiation (`add(5, 3)`)
3. Includes proper error handling for division by zero
4. Prints results in a formatted manner

Task 1:

Implement the calculator using templates.

- 1- add two numbers
- 2- subtract two numbers
- 3- divide two numbers
- 4- prime number
- 5- square root of number

Task 2

Implement the following template functions:

1. `findMin(T a, T b)` - Returns the minimum of two values
2. `findMin(int a, int b)` - Returns the minimum of two values
3. `findMin(float a, float b)` - Returns the minimum of two values
4. `swap(T& a, T& b)` - Swaps two values (by reference)
5. `remainder(T value, T value)` - Returns the remainder
6. `findMaxInArray(T arr[], size_t size)` - Find the maximum element in an array
7. `searchArray(T arr[], size_t size, const T& target)` - Linear search for an element, return index or -1
8. `reverseArray(T arr[], size_t size)` - Reverse the elements of an array in-place
9. `arraySum(T arr[], size_t size)` - Calculate sum of array elements (for numeric types)

Hint:

Size of an array can be found using: `size_t size = sizeof(arr) / sizeof(arr[0]);`

Task 3:

We have a problem in C++ we cannot handle an integer number more than 8bytes. So the solution is to store the number (from input) as string and then to array;

1- Your task is to convert the string input to array; *str="1112129191991991910"* to

1	1	1	2	1	2	9	1	9	1	9	9	1	9	9	1	9	1	9	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2- Now take input for 2 arrays using above method and then add two arrays. For example

Array 1=

1	3	2	9	7	8	4	5	8	3
---	---	---	---	---	---	---	---	---	---

Array 2=

9	1	2	9	7	3	8	5	8	1
---	---	---	---	---	---	---	---	---	---

Resultant array=

1	0	4	5	9	5	2	3	1	6	4
---	---	---	---	---	---	---	---	---	---	---

Task 4: Unit Testing with Google Test

Create comprehensive Google Tests for the above tasks 1,2,3 while testing for:

- Input with integers, floats, doubles
- Input having positive and negative numbers.
- Input with zero values.
- Input where result is negative
- Test for floating point results
- Also handle cases such as same values, duplicate values, special characters, empty arrays etc.

Note: You are required to write test cases that handle most scenarios. Write test cases from the above list that are applicable for your written function.